

Sequence and text sequence built-in types (mutable and immutable)

1. What will be printed in the code below?

```
my_string = "abc"
my_list = ["a", "b", "c"]
my_tuple = ("a", "b", "c")

print(my_string[0])
print(my_string[1])
print(my_string[2])
print(my_string + "de")
print(my_list + ["d", "e"])
print(my_tuple + ("d", "e"))
print(my_tuple + ("d",)) # Note that when defining a
tuple with only one element, you still need to add
a comma to show that it is a tuple.
```

a
b
c
abcde
a,b,c,d,e
a,b,c,d,e
a,b,c,d

2. Which of the following will result in an error?

```
my_string = "abc"
my_list = ["a", "b", "c"]
my_tuple = ("a", "b", "c")

☐ print(my_string + "de")
☐ print(my_list + ["d", "e"])
☐ print(my_tuple + ("d", "e"))
☒ print(my_tuple + ("d")) because ("d") is a string, not a tuple.
☐ print(my_string + my_tuple[0])
☒ print(my_string + my_list[3]) index 3 don't exist
☒ print(my_list + my_tuple) because you cannot directly concatenate a list and tuple.
☐ print(my_list + list(my_tuple))
☐ print(tuple(my_list) + my_tuple)
☐ print(str(my_list) + str(my_tuple))
```

Non-mandatory exercises: G

3. What will be printed?

```
my_string = "abc"
my_list = ["a", "b", "c"]
my_tuple = ("a", "b", "c")

new_string = my_string + my_tuple[0]
print(new_string)
print(my_string)

new_list = my_list + [my_string[0], my_list[0],
    my_tuple[0]]
print(new_list)
print(my_list)

new_tuple = my_tuple + (my_list[2],)
print(new_tuple)
print(my_tuple)
```

abca
abc
a,b,c,a,a,a
a,b,c
a,b,c,c
a,b,c

4. What will be printed?

```
my_string = "abc"
my_list = ["a", "b", "c"]
my_tuple = ("a", "b", "c")

my_string = my_string + my_tuple[0]
print(my_string)

my_list = my_list + [my_string[0], my_list[0],
    my_tuple[0]]
print(my_list)

my_tuple = my_tuple + (my_list[2],)
print(my_tuple)
```

abca
a,b,c,a,a,a
a,b,c,c

Non-mandatory exercises: G

5. Which of the following will result in an error?

```
my_string = "abc"
my_list = ["a", "b", "c"]
my_tuple = ("a", "b", "c")
```

- ☒ `my_string[0] = "A"` strings are immutable
- ☐ `my_list[0] = "A"`
- ☒ `my_tuple[0] = "A"` tuples are immutable
- ☐ `my_string = "A" + my_string[1:]`
- ☐ `my_list = ["A"] + my_list[1:]`
- ☐ `my_tuple = ("A",) + my_tuple[1:]`

6. All four code snippets below attempt to print abCde. Which one of them succeeds?

```
my_string = "abcde"
my_string = my_string[:2] + "C" + my_string[3:]
print(my_string)
```

```
my_string = "abcde"
my_string[2] = "C"
print(my_string)
```

strings immutable

```
my_string = "abcde"
my_string = my_string[:2] + my_string[2].upper() +
    my_string[3:]
print(my_string)
```

produces abCde because it replaces the character using upper().

```
my_string = "abcde"
my_list = list(my_string)
my_list[2] = my_list[2].upper()
my_string = "".join(list(my_list))
print(my_string)
```

it will also produce abCde
